



VIT[®]

BHOPAL

CODEVAULT

DSA Problem Tracking and Progress Management System

A Console-Based Java Application

Project Report

Submitted By: Shubh Gupta

Registration Number: 25BAI10562

Programme:

B.Tech Computer Science and Engineering – Artificial Intelligence and Machine Learning

University:

VIT Bhopal University

Course:

Programming In Java

Academic Year:

2026–2027

1. INTRODUCTION

Data Structures and Algorithms (DSA) are an essential part of computer science education and software development. Students regularly practice DSA problems to improve their problem-solving skills, prepare for technical interviews participate in coding competitions, and strengthen their understanding of fundamental algorithms and data structures.....

During DSA preparation, students solve problems belonging to different topics such as Arrays, Strings, Linked Lists, Stacks, Queues, Trees, Graphs, Dynamic Programming, Sorting, and Searching. As the number of solved and attempted problems increases, manually maintaining a record of these problems becomes difficult.

A student may have many questions in different stages such as solved, unsolved, or attempted multiple times. It can also become difficult to determine which topics have been practiced sufficiently and which topics require more attention.

CodeVault is a console-based Java application designed to provide a structured solution to this problem. It avjuncts as a personal DSA problem tracker where users can maintain a collection of coding problems, track their solving status, record attempts, search and filter problems, view progress, analyze statistics, and receive problem recommendations.

The project is implemented entirely using Core Java and is designed to run directly from the command line. The system follows a modular object-oriented design in which different responsibilities are divided among multiple Java classes.

The project demonstrates practical application of Java concepts such as Object-Oriented Programming, classes and objects, encapsulation, collections, file handling, exception handling, input validation, modular programming, and basic algorithmic processing.

The primary objective of CodeVault is not to solve DSA problems automatically. Instead, it manages and analyzes the user's DSA practice journey.

2. PROBLEM STATEMENT

Students who practice Data Structures and Algorithms often use multiple platforms and maintain their progress through notebooks, spreadsheets, bookmarks, or memory.

This approach creates several difficulties:

1. There is no centralized record of all practiced problems.
2. It becomes difficult to distinguish between solved and unsolved problems.
3. Students may forget how many attempts they made on a problem.
4. Finding problems belonging to a particular topic can become time-consuming.
5. Tracking overall solving progress manually requires additional effort.
6. Students may not have a simple way to identify weaker DSA topics.
7. Choosing the next problem to practice can become difficult when many problems are available.

For example, a student may have the following practice data:

Array	→ 10 problems
String	→ 8 problems
Linked List	→ 6 problems
Tree	→ 7 problems
Graph	→ 5 problems
DP	→ 3 problems

Without an organized tracking system, it is difficult to analyze this information and understand the student's progress.

Therefore, CodeVault is proposed as a lightweight console-based application that stores DSA problem information and provides tools for tracking, searching, filtering, analyzing, and improving the user's practice.

3. OBJECTIVES

The main objectives of CodeVault are:

3.1 DSA Problem Management

To provide a centralized system for adding and viewing DSA problems.

3.2 Problem Organization

To organize problems according to their topic and difficulty.

3.3 Progress Tracking

To maintain whether a particular problem has been solved or remains unsolved.

3.4 Attempt Tracking

To record the number of attempts made by the user on individual problems.

3.5 Search and Filtering

To allow users to quickly search for problems and filter them based on topic and difficulty.

3.6 Statistical Analysis

To calculate overall progress and topic-wise statistics from the stored problem data.

3.7 Recommendation

To analyze the user's practice information and recommend an appropriate problem for further practice.

3.8 User Management

To provide registration, login, logout, and profile functionality.

3.9 Application of Java Concepts

To implement a practical application using Core Java concepts including OOP, Collections, File I/O, Exception Handling, and modular programming.

3.10 Persistent Data

To maintain application data using local storage so that information can be loaded when the application is executed again.

4. FUNCTIONAL REQUIREMENTS

The CodeVault system consists of multiple functional modules. The major modules are User Management, Problem Management, Progress Tracking, Statistics and Analytics, and Recommendation.

4.1 User Management Module

The User Management module is responsible for managing user accounts and authentication.

Functions

- User registration
- User login
- User logout
- Profile viewing
- User validation

Registration

A new user provides:

Name

Username

Password

The system validates the entered information and creates the user account.

Login

The user enters:

Username

Password

The system verifies the credentials and allows access to the CodeVault dashboard if the credentials are valid.

Profile

The user can view basic account and progress information through the profile section.

5. PROBLEM MANAGEMENT MODULE

The Problem Management module is the primary module of CodeVault.

It allows the user to maintain their DSA problem collection.

5.1 Add Problem

The user can add a new DSA problem by providing:

- Problem title
- Topic
- Difficulty

Each problem receives a unique problem ID.

Example:

Problem ID: 101
Title: Two Sum
Topic: Array
Difficulty: Easy
Status: Unsolved
Attempts: 0

5.2 View All Problems

The user can view all problems stored in the system.

Example:

ID	Title	Topic	Difficulty
Status			

101	Two Sum	Array	Easy
Solved			
102	Binary Search	Searching	Easy
Solved			
103	Number of Islands	Graph	Medium
Unsolved			
104	LCS	DP	Hard
Unsolved			

5.3 Search Problem

The search functionality allows users to search for problems using a title or keyword.

For example:

Search: Binary

The system can display matching problems such as:

Binary Search

Binary Tree Traversal

5.4 Filter by Topic

Users can filter problems according to DSA topic.

Supported topics include:

1. Array
2. String
3. Linked List
4. Stack
5. Queue
6. Tree
7. Graph
8. DP
9. Sorting
10. Searching

This allows users to focus on a specific area of DSA.

5.5 Filter by Difficulty

Problems can also be filtered according to difficulty.

The supported difficulty levels are:

Easy

Medium

Hard

This allows users to select problems according to their current preparation level.

6. PROGRESS TRACKING MODULE

The Progress Tracking module maintains information about the user's solving activity.

Each problem contains a solving status.

The possible states are:

Solved

Unsolved

The system also maintains the number of attempts made for each problem.

For example:

Problem: Number of Islands

Topic: Graph

Difficulty: Medium

Attempts: 3

Status: Unsolved

If the user successfully solves the problem, the status can be changed:

Problem: Number of Islands

Attempts: 4

Status: Solved

The user can also mark a problem as unsolved if required.

7. STATISTICS AND ANALYTICS MODULE

The Statistics module processes the stored problem information and generates useful information about the user's DSA practice.

The statistics module can calculate:

- Total number of problems
- Number of solved problems
- Number of unsolved problems
- Completion percentage
- Total attempts
- Topic-wise performance
- Difficulty-wise distribution
- Weak areas

7.1 Completion Percentage

The completion percentage can be calculated using:

$$\text{Completion Percentage} = (\text{Solved Problems} / \text{Total Problems}) \times 100$$

For example:

Total Problems = 20

Solved Problems = 12

$$\text{Completion Percentage} = (12 / 20) \times 100$$

= 60%

7.2 Topic-Wise Statistics

The application can analyze progress for individual topics.

Example:

Topic	Total	Solved
-----	-----	-----
Array	10	8
String	8	6
Linked List	6	4
Tree	7	3
Graph	5	1
DP	3	0

This allows the user to understand how their practice is distributed.

8. RECOMMENDATION ENGINE

The Recommendation Engine is one of the analytical components of CodeVault.

Its purpose is to suggest an unsolved problem based on the user's existing progress.

The recommendation process can be represented as:

User Problem Data

↓

Analyze Topic Performance

↓

Identify Weak/Low-Progress Topic

↓

Find Unsolved Problems

↓

Select Suitable Problem

↓

Display Recommendation

For example:

Array → 80% solved

String → 75% solved

Tree → 43% solved

Graph → 20% solved

DP → 0% solved

The system can identify DP or Graph as an area requiring additional practice.

It can then recommend an unsolved problem from that topic.

Example:

Recommended Problem:

Longest Common Subsequence

Topic:

Dynamic Programming

Difficulty:

Medium

The recommendation system is **rule-based**. It does not use machine learning or artificial intelligence.

9. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the qualities and constraints of the CodeVault application.

9.1 Usability

The application should provide a simple and understandable command-line interface.

The user should be able to understand the available operations through numbered menus.

Example:

1. Add Problem
2. View All Problems
3. Search Problem
4. Filter by Topic
5. Filter by Difficulty
6. Mark Problem Solved
7. Record Attempt
8. Mark Problem Unsolved
9. View Progress
10. View Statistics
11. Get Recommendation
12. Profile
13. Logout

9.2 Performance

The application should perform searching, filtering, statistics calculation, and other operations efficiently for the expected scale of a personal DSA problem collection.

Since CodeVault is intended for individual student use, the expected dataset size is relatively small.

9.3 Reliability

The application should continue operating correctly when users provide normal invalid input.

For example, an invalid menu choice should generate an appropriate message rather than causing an application crash.

9.4 Maintainability

The project follows a modular architecture.

Different responsibilities are implemented in separate classes, making it easier to modify or extend individual components without affecting the entire application.

9.5 Error Handling

The system validates user input and handles common runtime errors.

Examples include:

- Invalid menu selection
- Invalid problem ID
- Invalid topic
- Invalid difficulty
- Invalid login credentials
- Empty input
- File-related errors

9.6 Resource Efficiency

CodeVault is a lightweight command-line application.

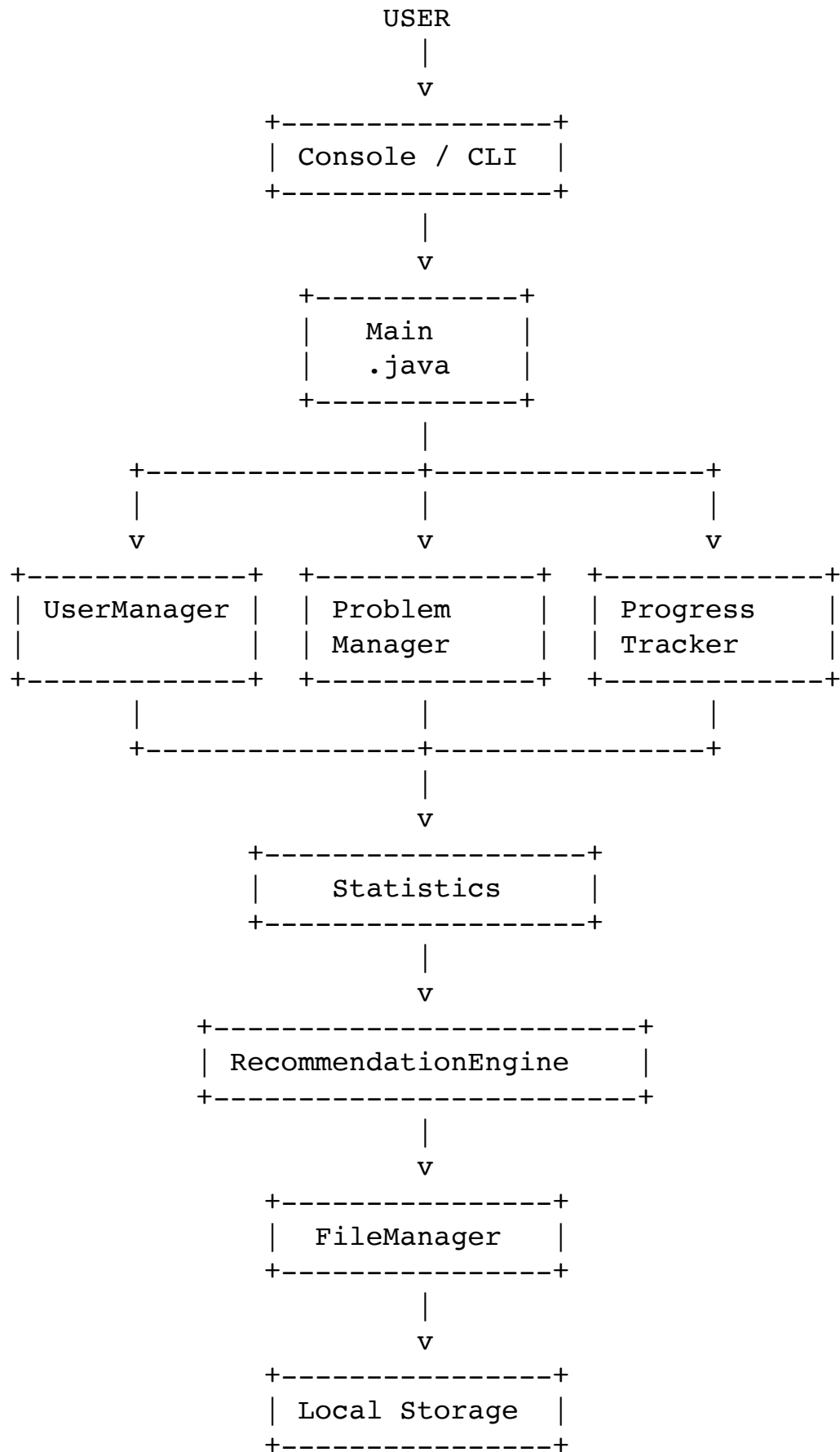
It does not require a web server, database server, or graphical environment. Local storage and standard Java data structures are sufficient for its intended use.

10. SYSTEM ARCHITECTURE

CodeVault follows a modular architecture.

The user interacts with the system through the console. The `Main` class controls the overall application flow and communicates with manager and service classes.

10.1 High-Level Architecture



10.2 Architecture Components

Main

Controls application execution, menus, user interaction, and overall workflow.

User

Represents a CodeVault user.

Problem

Represents an individual DSA problem.

UserManager

Handles user-related operations such as registration and authentication.

ProblemManager

Handles problem-related operations such as adding, viewing, searching, and filtering problems.

ProgressTracker

Handles solving status and attempt information.

Statistics

Processes problem data and calculates progress and performance information.

RecommendationEngine

Analyzes user progress and generates recommendations.

FileManager

Handles persistent storage and retrieval of application data.

InputValidator

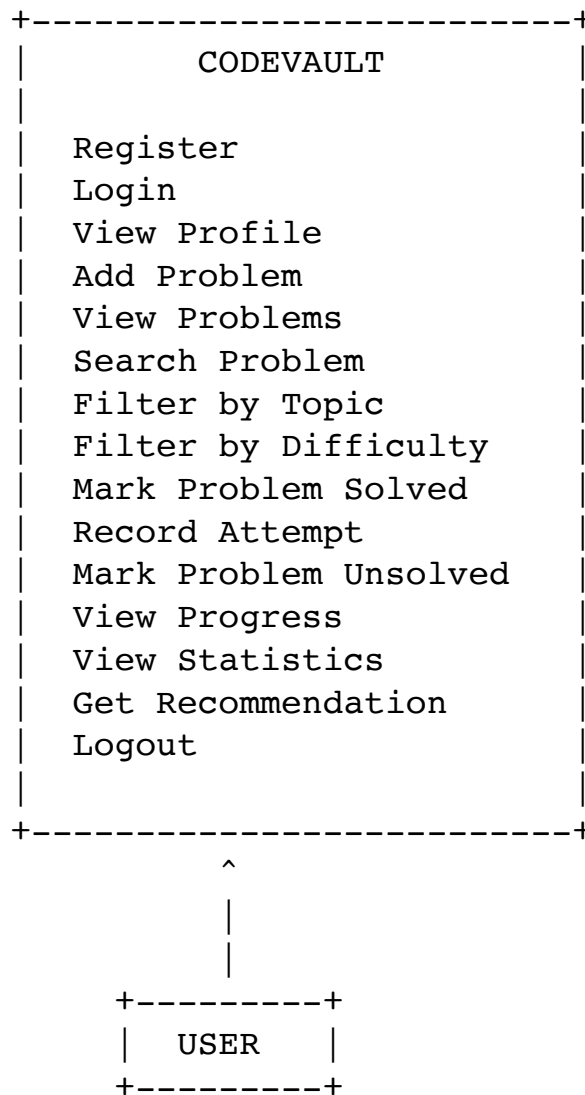
Validates user input before it is processed by the application.

11. DESIGN DIAGRAMS

11.1 USE CASE DIAGRAM

The primary actor of CodeVault is the **User/Student**.

The user interacts with the following system functions:



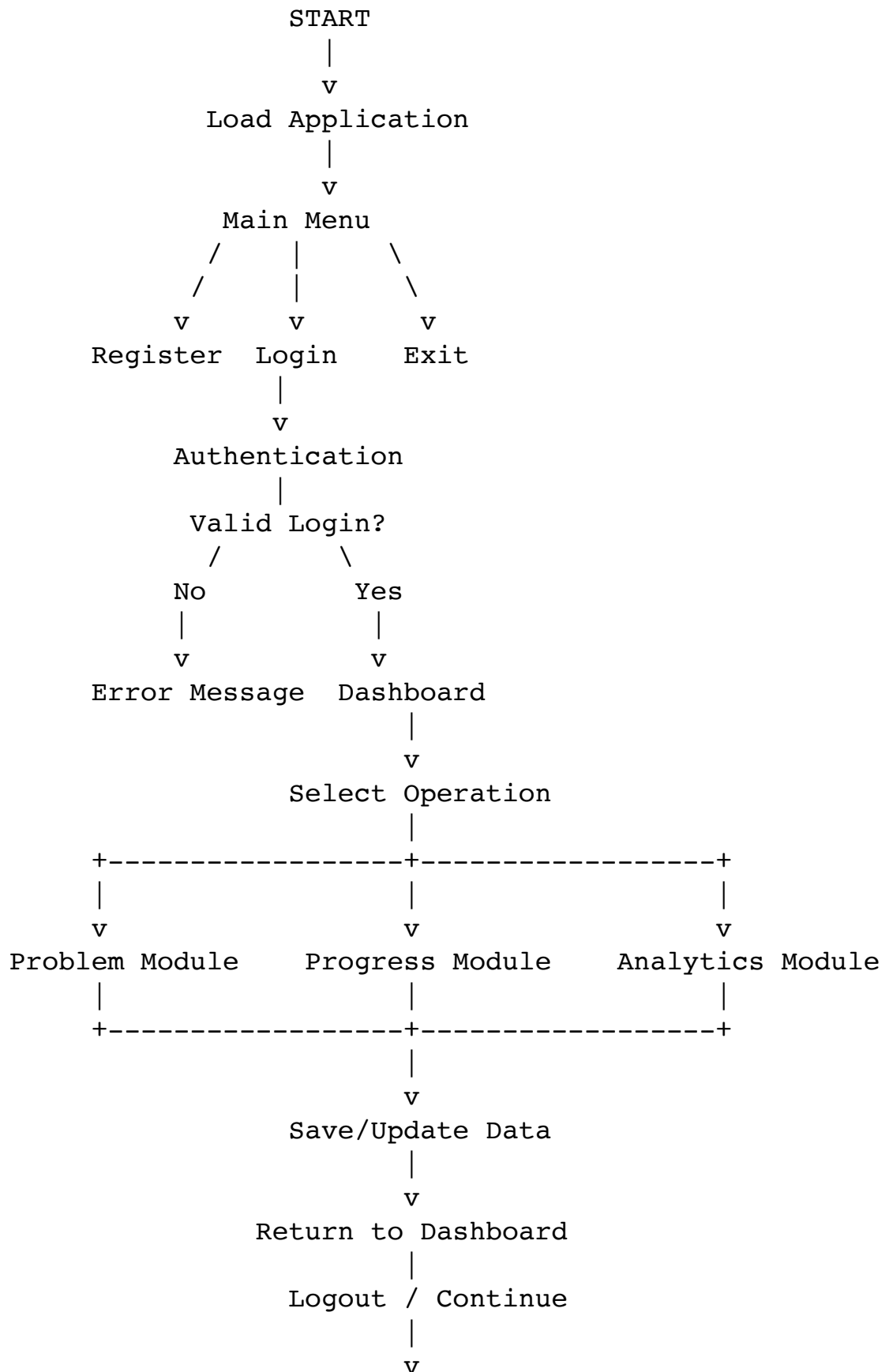
Explanation

The Use Case Diagram represents the interaction between the user and CodeVault.

The user is responsible for initiating all major operations. Authentication-related operations are handled before access to user-specific features is provided.

12. WORKFLOW DIAGRAM

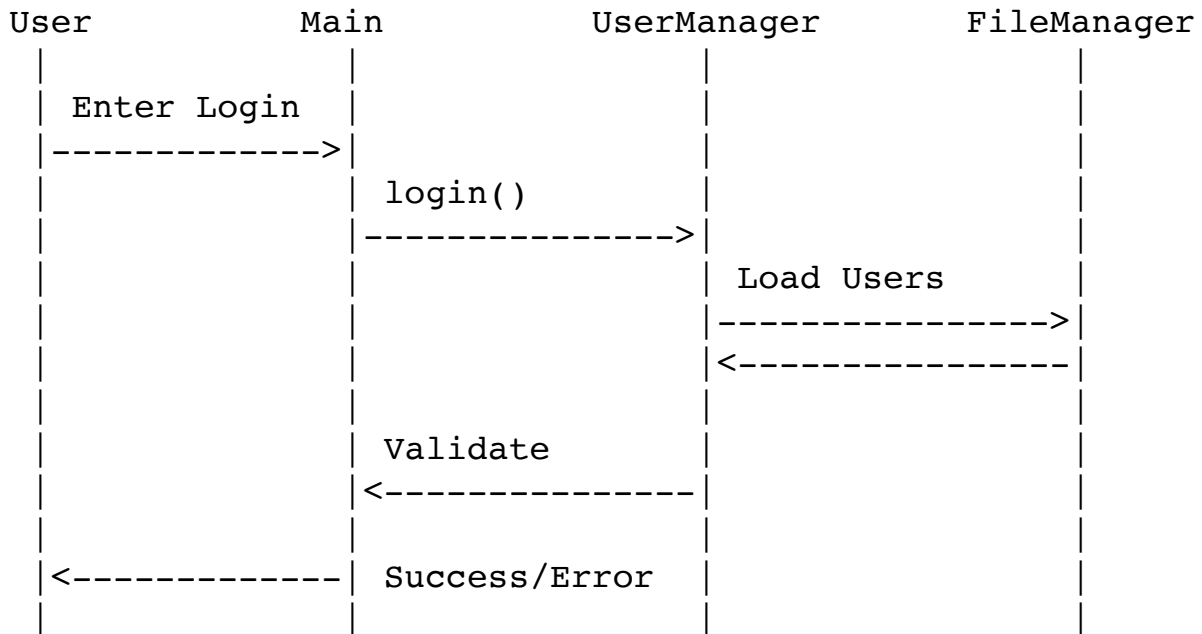
The overall workflow of CodeVault is:



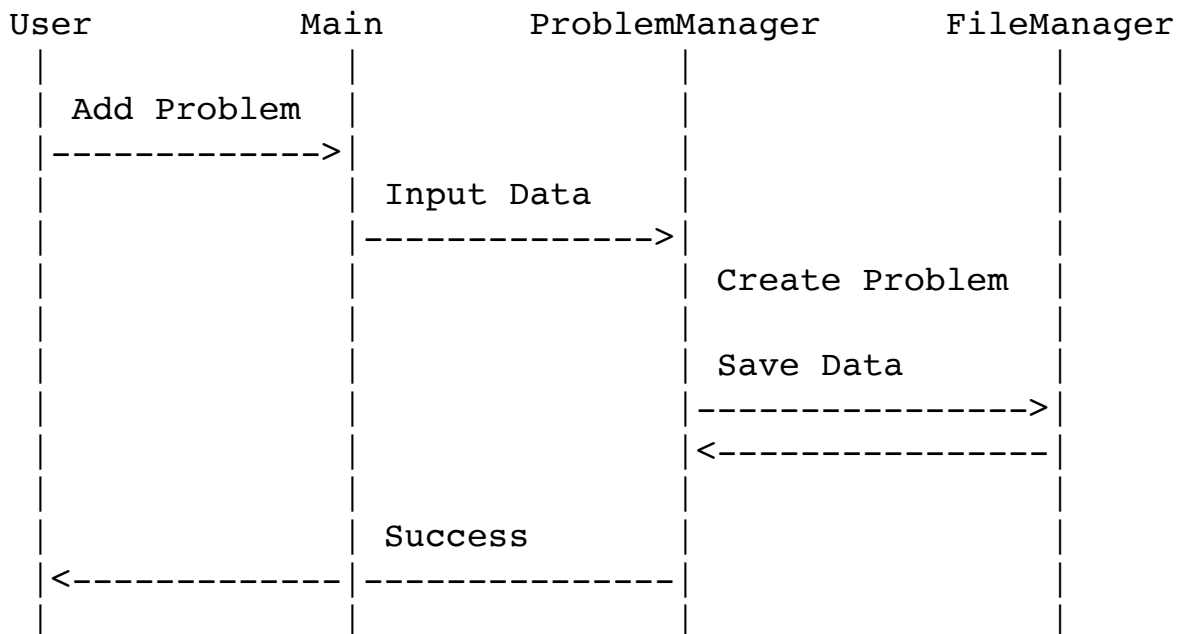
END

13. SEQUENCE DIAGRAM

13.1 Login Sequence



13.2 Add Problem Sequence

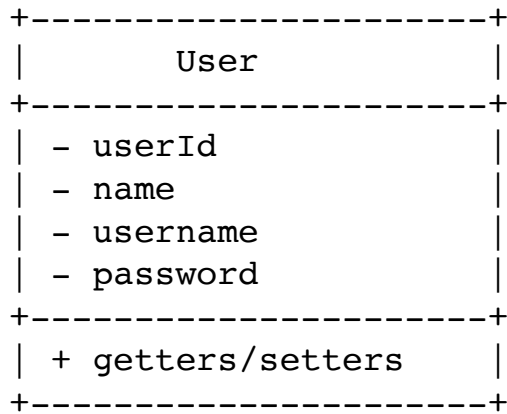


13.3 Mark Problem Solved Sequence

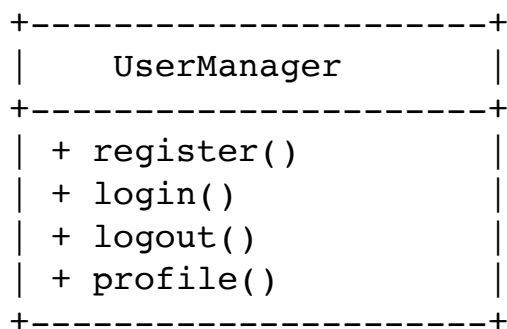
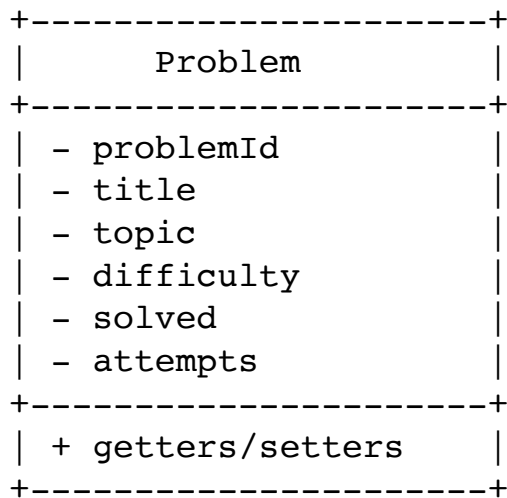
```
User
|
| Enter Problem ID
v
Main
|
| markSolved(id)
v
ProgressTracker
|
| Find Problem
v
ProblemManager
|
| Update Status
v
Problem
|
| solved = true
v
FileManager
|
| Save Updated Data
v
Main
|
| Display Success
v
User
```

14. CLASS DIAGRAM / COMPONENT DIAGRAM

The project consists of the following major classes:



|
|
v



```
+-----+
|   ProblemManager   |
+-----+
| + addProblem()      |
| + viewProblems()    |
| + searchProblem()   |
| + filterByTopic()   |
| + filterByDifficulty() |
+-----+
```

```
+-----+
|   ProgressTracker  |
+-----+
| + markSolved()      |
| + markUnsolved()    |
| + recordAttempt()   |
| + calculateProgress() |
+-----+
```

```
+-----+
|   Statistics        |
+-----+
| + totalProblems()   |
| + solvedProblems()  |
| + topicStatistics() |
| + completionRate()  |
+-----+
```

```
+-----+
| RecommendationEngine |
+-----+
| + recommendProblem() |
| + findWeakTopic()    |
+-----+
```

```
+-----+
|   FileManager       |
+-----+
| + saveData()         |
| + loadData()         |
+-----+
```

```

+-----+
|   InputValidator   |
+-----+
| + validateInput()  |
| + validateTopic()  |
| + validateDifficulty() |
+-----+

```

```

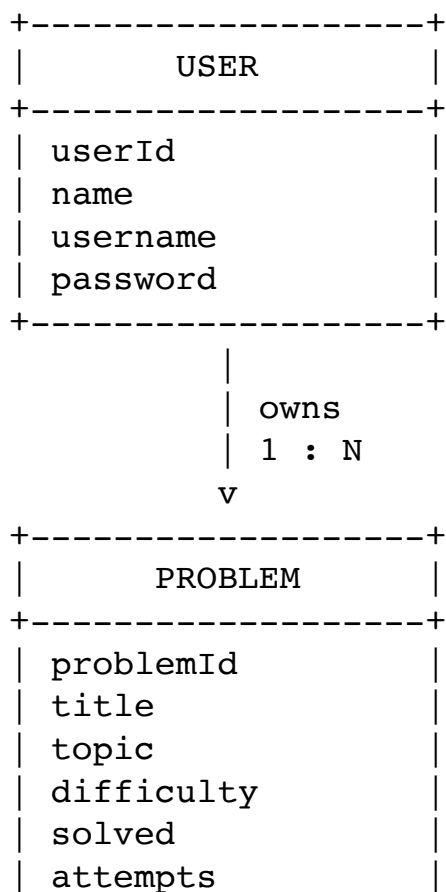
+-----+
|           Main      |
+-----+
| + main()             |
| + applicationMenu() |
+-----+

```

15. STORAGE DESIGN

CodeVault uses local file-based storage instead of a relational database.

The conceptual relationship between users and their problems can be represented as:



+-----+

One user can have multiple DSA problems.

The actual implementation uses Java objects and local file storage rather than a SQL database.

16. DESIGN DECISIONS AND RATIONALE

16.1 Choice of Java

Java was selected because the project is intended to demonstrate Core Java programming concepts.

Java provides built-in support for:

- Object-Oriented Programming
- Collections
- File I/O
- Exception Handling
- Encapsulation
- Modular programming

16.2 Choice of Console Interface

The project is intentionally designed as a command-line application.

A CLI provides a simple environment in which the main focus can remain on Java programming concepts and application logic.

It also satisfies the requirement that the project should be executable directly through the command line.

16.3 Modular Class Structure

The application is divided into multiple classes rather than placing all functionality inside `Main.java`.

This separation of responsibilities makes the code easier to understand and maintain.

For example:

UserManager	→ User operations
ProblemManager	→ Problem operations
Statistics	→ Statistical processing

FileManager → Storage

16.4 Use of Collections

Java collections are used to store and process multiple objects.

An `ArrayList` is appropriate for maintaining a dynamic collection of problems because the number of problems can increase during application usage.

A `HashMap` can be used for efficient topic-wise statistical calculations.

16.5 File-Based Storage

A local file-based approach was selected because CodeVault is designed as a lightweight personal application.

It avoids the need for:

- Database server
- Network connection
- Database credentials
- External infrastructure

The application can therefore be executed directly from a terminal.

16.6 Rule-Based Recommendation

A rule-based recommendation approach was selected instead of machine learning.

The purpose is to demonstrate logical processing of application data without introducing unnecessary complexity.

The recommendation engine can analyze topic-wise solving progress and select an unsolved problem from a lower-progress topic.

17. IMPLEMENTATION DETAILS

17.1 Project Structure

The project contains the following Java files:

```
CodeVault/  
|
```

```
├── Main.java
├── User.java
├── Problem.java
├── ProblemManager.java
├── UserManager.java
├── ProgressTracker.java
├── RecommendationEngine.java
├── Statistics.java
├── FileManager.java
└── InputValidator.java
```

The project is organized around separate responsibilities.

17.2 Main.java

`Main.java` serves as the entry point of the application.

It:

- Starts the application
- Displays menus
- Accepts user choices
- Directs requests to appropriate classes
- Controls login and logout flow

The application is started using the Java `main ( )` method.

17.3 User.java

The `User` class represents an individual CodeVault user.

It stores information such as:

```
User ID
Name
Username
Password
```

The class demonstrates the concept of encapsulation by maintaining user-related data inside the object.

17.4 Problem.java

The `Problem` class represents an individual DSA problem.

Its important attributes include:

Problem ID

Title

Topic

Difficulty

Solved Status

Attempts

Each problem is represented as an independent Java object.

17.5 UserManager.java

`UserManager` manages user-related operations.

Its responsibilities include:

- Registration
- Login
- User validation
- Logout
- Profile-related operations

Separating these operations from `Main.java` improves modularity.

17.6 ProblemManager.java

`ProblemManager` handles DSA problem operations.

Its responsibilities include:

- Adding problems
- Viewing problems
- Searching problems
- Filtering by topic
- Filtering by difficulty

It acts as the primary controller for problem-related data.

17.7 ProgressTracker.java

`ProgressTracker` manages progress-related information.

It handles:

- Marking problems solved
- Marking problems unsolved
- Recording attempts
- Calculating progress

17.8 Statistics.java

`Statistics` processes the problem collection to calculate useful information.

Examples include:

Total Problems
Solved Problems
Unsolved Problems
Completion Percentage
Topic-wise Statistics

17.9 RecommendationEngine.java

`RecommendationEngine` analyzes the user's existing problem data.

It identifies areas with comparatively lower progress and selects an appropriate unsolved problem.

The process is based on predefined logical rules rather than machine learning.

17.10 FileManager.java

`FileManager` handles data persistence.

It provides methods for:

Saving data
Loading data

This allows information to remain available between different executions of the application.

17.11 InputValidator.java

`InputValidator` is responsible for validating user input.

Validation is important in a console application because users may enter unexpected values.

Examples include:

Invalid menu number
Invalid topic
Invalid difficulty
Invalid ID
Empty input

18. INPUT AND OUTPUT STRUCTURE

CodeVault follows a straightforward input-processing-output model.

INPUT

|

v

User enters choice/data

|

v

InputValidator

|

v

Manager/Service Class

|

v

Processing

|

v

FileManager / Data

|

v

OUTPUT

For example:

Input

Problem title: Two Sum

Topic: Array

Difficulty: Easy

Processing

Validate input

Create Problem object

Assign ID

Store problem

Output

Problem added successfully!

19. SAMPLE APPLICATION EXECUTION

A typical execution of CodeVault begins with:

```
=====
                        CODEVAULT
                Track. Solve. Improve.
=====
1. Register
2. Login
3. Exit
Enter choice:
After successful login:

=====
Welcome, Shubh!
1. Add Problem
2. View All Problems
3. Search Problem
4. Filter by Topic
5. Filter by Difficulty
6. Mark Problem Solved
7. Record Attempt
8. Mark Problem Unsolved
9. View Progress
10. View Statistics
11. Get Recommendation
12. Profile
13. Logout
Enter choice:
If no problems have been added:

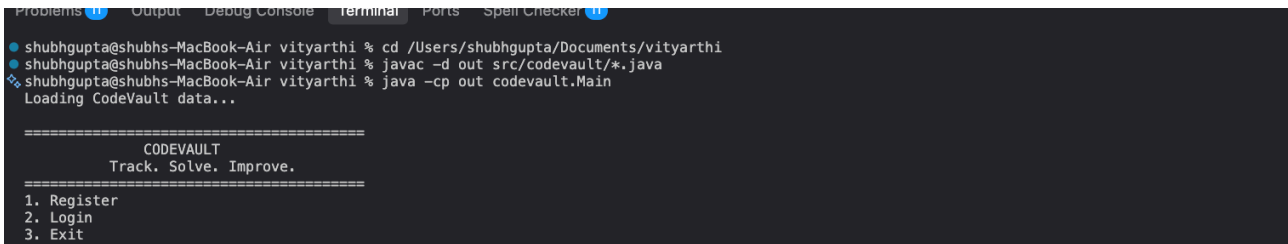
No problems found.
After adding problems, the user can view and manage them through the available options.
```

20. SCREENSHOTS / RESULTS

This section should contain screenshots captured from the actual final execution of the application.

The following screenshots are recommended.

20.1 Main Menu



```
Problems Output Debug Console Terminal Ports Spell Checker
shubhgupta@shubhs-MacBook-Air vityarthi % cd /Users/shubhgupta/Documents/vityarthi
shubhgupta@shubhs-MacBook-Air vityarthi % javac -d out src/codevault/*.java
shubhgupta@shubhs-MacBook-Air vityarthi % java -cp out codevault.Main
Loading CodeVault data...

=====
CODEVAULT
Track. Solve. Improve.
=====
1. Register
2. Login
3. Exit
```

Caption:

Figure 1: CodeVault Main Menu

The screenshot should show the initial CodeVault interface containing registration, login, and exit options.

20.2 User Registration



```
=====
CODEVAULT
Track. Solve. Improve.
=====
1. Register
2. Login
3. Exit
Enter choice: 1
Enter name: Shubh
Enter username: shubh_16.1
Enter password (minimum 4 characters): shubh
Registration successful!
```

Caption:

Figure 2: User Registration

The screenshot should demonstrate the process of creating a new CodeVault account.

20.3 Successful Login



```
=====
CODEVAULT
Track. Solve. Improve.
=====
1. Register
2. Login
3. Exit
Enter choice: 2
Username: shubh_16.1
Password: shubh

=====
Welcome, Shubh!
1. Add Problem
2. View All Problems
3. Search Problem
4. Filter by Topic
5. Filter by Difficulty
6. Mark Problem Solved
7. Record Attempt
8. Mark Problem Unsolved
9. View Progress
```

Caption:

Figure 3: Successful User Login

The screenshot should show the welcome message and user dashboard after successful authentication.

20.4 Add Problem

```
=====
Welcome, Shubh!
1. Add Problem
2. View All Problems
3. Search Problem
4. Filter by Topic
5. Filter by Difficulty
6. Mark Problem Solved
7. Record Attempt
8. Mark Problem Unsolved
9. View Progress
10. View Statistics
11. Get Recommendation
12. Profile
13. Logout
Enter choice: 1
Problem title: sum of two number
Topics: Array, String, Linked List, Stack, Queue, Tree, Graph, DP, Sorting, Searching
Topic: 1
Invalid topic.
Topics: Array, String, Linked List, Stack, Queue, Tree, Graph, DP, Sorting, Searching
Topic: array
Difficulty (Easy/Medium/Hard): hard

Problem added successfully! Problem ID: 1
=====
```

Caption:

Figure 4: Adding a DSA Problem

The screenshot should demonstrate entering the problem title, topic, and difficulty.

20.5 View Problems

```
=====
Welcome, Shubh!
1. Add Problem
2. View All Problems
3. Search Problem
4. Filter by Topic
5. Filter by Difficulty
6. Mark Problem Solved
7. Record Attempt
8. Mark Problem Unsolved
9. View Progress
10. View Statistics
11. Get Recommendation
12. Profile
13. Logout
Enter choice: 2

  ID  TITLE                TOPIC    DIFFICULTY  STATUS    ATTEMPTS
-----
  1   sum of two number     Array    Hard        Unsolved   Attempts: 0
=====
```

Caption:

Figure 5: Viewing Stored Problems

The screenshot should show the list of problems stored by the user.

20.6 Search / Filter

```
=====
Welcome, Shubh!
1. Add Problem
2. View All Problems
3. Search Problem
4. Filter by Topic
5. Filter by Difficulty
6. Mark Problem Solved
7. Record Attempt
8. Mark Problem Unsolved
9. View Progress
10. View Statistics
11. Get Recommendation
12. Profile
13. Logout
Enter choice: 4
Topics: Array, String, Linked List, Stack, Queue, Tree, Graph, DP, Sorting, Searching
Topic: Array

ID  TITLE                                TOPIC  DIFFICULTY  STATUS  ATTEMPTS
-----
1   sum of two number                    Array   Hard        Unsolved  Attempts: 0
=====
```

Caption:

Figure 6: Searching or Filtering Problems

The screenshot should demonstrate searching or filtering the stored problems.

21. TESTING APPROACH

Testing was performed to verify that the different modules of CodeVault work correctly and that invalid user inputs are handled appropriately.

The testing approach includes functional testing and validation testing.

21.1 Functional Testing

Each major feature is tested individually.

Test ID	Test Case	Input/Action	Expected Result
TC01	Registration	New valid user	Registration successful
TC02	Duplicate Registration	Existing username	Registration rejected
TC03	Valid Login	Correct credentials	Dashboard displayed
TC04	Invalid Login	Incorrect credentials	Error message displayed
TC05	Add Problem	Valid problem details	Problem added
TC06	View Problems	Select view option	Stored problems displayed
TC07	Search Problem	Valid keyword	Matching problems displayed
TC08	Filter Topic	Select topic	Relevant problems displayed
TC09	Filter Difficulty	Select difficulty	Relevant problems displayed
TC10	Mark Solved	Valid problem ID	Problem becomes solved
TC11	Record Attempt	Valid problem ID	Attempt count increases
TC12	Mark Unsolved	Valid problem ID	Problem becomes unsolved
TC13	Progress	Existing problem data	Progress calculated
TC14	Statistics	Existing problem data	Statistics displayed
TC15	Recommendation	Existing unsolved problems	Recommendation displayed
TC16	Profile	Logged-in user	Profile displayed
TC17	Logout	Select logout	User returned to main menu
TC18	Invalid Input	Invalid menu choice	Error handled
TC19	Invalid ID	Non-existing problem ID	Appropriate error
TC20	Persistence	Restart application	Stored data loaded

22. VALIDATION TESTING

The application must handle incorrect input safely.

Examples include:

Invalid Menu Choice

Enter choice: 99

Invalid choice.
Please try again.

Invalid Problem ID

Enter Problem ID: 9999

Problem not found.

Invalid Topic

Enter Topic: xyz

Invalid topic.

Invalid Difficulty

Enter Difficulty: Extreme

Invalid difficulty.

Invalid Password

Password must contain at least 4 characters.

These checks improve application reliability and usability.

23. CHALLENGES FACED

Several challenges were encountered or considered during the development of CodeVault.

23.1 Designing a Modular Architecture

One of the primary challenges was determining how to divide the application into multiple meaningful classes.

Instead of implementing all functionality inside `Main.java`, responsibilities were distributed across classes such as:

UserManager

ProblemManager

ProgressTracker

Statistics

RecommendationEngine

FileManager

InputValidator

This improved the organization of the project.

23.2 Handling Console Input

Console applications depend entirely on user input.

Users can enter unexpected values such as:

Letters instead of numbers

Invalid IDs

Empty values

Unsupported topics

Unsupported difficulty levels

Therefore, input validation became an important part of the implementation.

23.3 Maintaining Data Persistence

Another challenge was ensuring that application information does not disappear when the program is closed.

The `FileManager` class was introduced to handle saving and loading application data.

23.4 Calculating Statistics

Statistics require processing multiple problem objects.

For example, calculating the number of solved problems requires iterating through the stored collection and checking each problem's status.

Similarly, topic-wise statistics require grouping problems according to their topics.

23.5 Designing the Recommendation Logic

The recommendation system required defining logical rules for selecting a suitable problem.

The system considers information such as topic progress and solving status to identify an appropriate unsolved problem.

24. LEARNINGS AND KEY TAKEAWAYS

The development of CodeVault provided practical experience in several Java programming concepts.

24.1 Object-Oriented Programming

The project demonstrated how real-world entities can be represented using classes and objects.

For example:

User

Problem

are represented as Java classes.

24.2 Encapsulation

Encapsulation was applied by keeping object data within classes and providing controlled methods for accessing and modifying the information.

24.3 Java Collections

Collections such as `ArrayList` provide dynamic storage for problem objects.

They simplify operations such as adding, searching, filtering, and iterating through problems.

24.4 File Handling

The project provided practical experience with persistent data storage using Java file handling.

24.5 Exception Handling

The project demonstrated how exceptions and invalid inputs can be handled without allowing the application to terminate unexpectedly.

24.6 Modular Programming

Separating the project into multiple classes demonstrated the importance of assigning each component a specific responsibility.

24.7 Algorithmic Thinking

The statistics and recommendation components required logical processing of stored information.

This helped demonstrate how simple algorithms can be used to extract useful information from application data.

24.8 Software Development Process

The project also provided experience with:

Requirement Analysis



System Design
↓
Implementation
↓
Testing
↓
Documentation
↓
Version Control

25. FUTURE ENHANCEMENTS

Although CodeVault is designed as a console-based Java application, several improvements can be implemented in future versions.

25.1 Database Integration

The current local storage system can be replaced or supplemented with a relational database such as MySQL.

JDBC can be used to connect Java with the database.

This would make the application more suitable for larger datasets.

25.2 Graphical User Interface

A future version could use JavaFX or another Java GUI framework to provide a graphical interface.

Possible features could include:

Dashboard

Progress Charts

Topic Graphs

Problem Tables

Interactive Filters

However, the current project intentionally remains CLI-based.

25.3 Advanced Recommendation System

The current recommendation engine uses predefined rules.

A future version could analyze additional information such as:

- Historical attempts

- Average solving time
- Difficulty progression
- Topic performance
- Recent activity

This could provide more personalized recommendations.

25.4 Coding Streak Tracking

A future version could track daily coding activity.

For example:

Cugrrenyt Streak: 7 days

Longest Streak: 21 days

25.5 Online Problem Integration

Future versions could integrate with coding platforms through supported APIs or data sources.

This could allow users to import problem information instead of manually entering it.

25.6 Difficulty Progression

The applicgation could automatically recommend increasingly difficult problems based on the user's solving history.

For example:

Easy → Medium → Hard

25.7 Cloud Synchronization

A future version could store user data remotely, allowing the same CodeVault account to be accessed from multiple devices.

26. CONCLUSION

CodeVault is a console-based DSA Problem Tracking and Progress Management System developed using Cjore Java.

The application addresses the practical problem of maintaining and analyzing DSA practice data. It provides user management, problem management, progress tracking, statistics, and recommendation functionality through a command-line interface.

The modular structure of the application separates responsibilities among different classes such as `UserManager`, `ProblemManager`, `ProgressTracker`, `Statistics`, `RecommendationEngine`, `FileManager`, and `InputValidator`.

The project demonstrates important Java concepts including Object-Oriented Programming, encapsulation, collections, file handling, exception handling, input validation, and modular software development.

The statistics and recommendation components provide an additional analytical layer beyond simple problem storage. Instead of merely recording problems, CodeVault processes the stored information to help users understand their practice progress and identify areas requiring additional attention.

The project also provides a foundation for future improvements such as database integration, graphical interfaces, advanced recommendation systems, online problem integration, streak tracking, and cloud synchronization.

Overall, CodeVault demonstrates how Core Java concepts can be combined to develop a practical, modular, and executable command-line application for managing a student's DSA preparation journey.

27. REFERENCES

1. Oracle Corporation, **Java Documentation**, Java Platform Documentation.
2. Oracle Corporation, **The Java Tutorials**, documentation related to Java classes, objects, collections, exceptions, and file handling.
3. Java Platform Standard Edition documentation for Collections Framework and Input/Output APIs.
4. Course lecture materials and notes provided as part of the Java programming curriculum.

5. Data Structures and Algorithms learning resources used during the development and testing of the project.
6. Git and GitHub documentation for source code management and version control.